

```
1 #include <stdio.h>
2
3 int main() {
4     // Write C code here
5     printf("Hello World!");
6
7     return 0;
8 }
9
```

LINGUAGEM E TÉCNICAS DE PROGRAMAÇÃO

Estrutura Condicional



Linguagem C

- Em C, a delimitação de blocos de código é feita por **chaves** `{ }`. E após cada comando é preciso inserir um `;`
- C possui **tipagem estática**: o tipo da variável deve ser definido no momento da declaração (`int`, `float`, `char`, etc.) e **não pode ser alterado** durante a execução.
- C requer **compilação**: o código-fonte é traduzido para código de máquina por um compilador antes de ser executado, gerando um arquivo executável. Isso tende a produzir programas mais rápidos. C é **case-sensitive**: diferencia letras maiúsculas e minúsculas. Por exemplo, `Palavra` e `palavra` são variáveis diferentes.
- C permite **programação de baixo nível**, com acesso direto à memória por meio de **ponteiros**, o que possibilita alto desempenho, mas exige mais cuidado para evitar erros.



Linguagem C

C possui **bibliotecas padrão** para diversas tarefas:

- Manipulação de entrada/saída (**stdio.h**).
- Manipulação de strings (**string.h**).
- Funções matemáticas (**math.h**).
- Controle de tempo (**time.h**).

Portabilidade: programas em C podem ser compilados em praticamente qualquer sistema operacional ou arquitetura de hardware, com poucas ou nenhuma modificação no código, desde que use apenas bibliotecas padrão.

Base para outras linguagens: C influenciou diretamente linguagens como C++, C#, Java, JavaScript, Go e muitas outras.

Linguagem estruturada: incentiva a programação usando funções e divisão lógica do código, facilitando organização e manutenção.



Atribuição de valores às variáveis

- Uma vez declarada, podemos atribuir **valores** às **variáveis**;

```
nome_da_variavel = expressão;
```

- A **expressão** é uma combinação entre **operadores** e **operandos** que deve ter seu valor avaliado antes de ser atribuído à variável



Atribuição de valores às variáveis

- Uma **expressão** é basicamente qualquer coisa que quando avaliado, resulta em um **valor**;
 - ❑ Exemplo:
 - $2 * (5 + 1) - 3 \rightarrow 9$
- Temos basicamente 3 tipos de expressões:
 - ❑ Aritmética
 - ❑ Relacional
 - ❑ Lógica
- Os **operandos** podem ser formados por valores imediatos, constantes, variáveis, ou outras expressões;



Operador	Significado	Exemplo	Verdadeiro se...
<	Menor que	$a < b$	a é estritamente menor que b
<=	Menor ou igual a	$a <= b$	a é menor ou igual a b
>	Maior que	$a > b$	a é estritamente maior que b
>=	Maior ou igual a	$a >= b$	a é maior ou igual a b
==	Igual a	$a == b$	a e b têm o mesmo valor
!=	Diferente de	$a != b$	a e b têm valores diferentes



Esses operadores permitem criar condições mais complexas, como mostra o exemplo seguinte, no qual se deseja saber se a divisão de x por 2 é maior do que o valor de y menos 3:

$$x/2 > y - 3$$



Uma expressão condicional pode utilizar operadores dos tipos matemático, relacional e/ou lógico.

```
01  x e maior ou igual a y?  
02      x >= y  
03  
04  x e maior do que y+2?  
05      x > y+2  
06  
07  x-5 e diferente de y+3?  
08      x-5 != y+3  
09  
10  x e maior do que y e menor do que z?  
11      (x > y) && (x < z)
```

Quando o compilador avalia uma condição, ele quer um valor de retorno (**verdadeiro** ou **falso**) para poder tomar a decisão. No entanto, essa expressão condicional não necessita ser uma expressão no sentido convencional.



Aritméticos

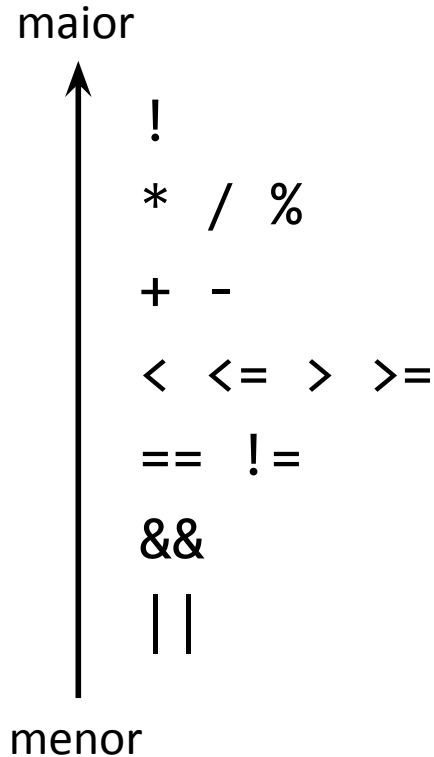
+	Adição	$a + b$
-	Subtração	$a - b$
*	Multiplicação	$a * b$
/	Divisão	a / b
%	Módulo (resto da divisão)	$a \% b$

Incremento/Decremento

++	Incremento (+1)	$a++$ ou $++a$
--	Decremento (-1)	$a--$ ou $--a$

Atribuição

=	Atribuição simples	$a = b$
+=	Soma e atribui	$a += 5$
-=	Subtrai e atribui	$a -= 5$
*=	Multiplica e atribui	$a *= 5$
/=	Divide e atribui	$a /= 5$



- Lembrando que, assim como na matemática, parênteses podem ser empregados para alterar a ordem de precedência das operações



Estrutura Condicional

Os programas escritos até o momento são programas sequenciais:

um comando é executado após o outro, do começo ao fim do programa, na ordem em que foram declarados no código-fonte.

Nenhum comando é ignorado.



Estrutura Condicional

- É uma estrutura de controle de fluxo que executa um ou vários comandos **SE** a **condição** resultar **Verdadeiro** (1) e, em alguns casos, executa um ou vários comandos **SE** resultar **Falso** (0).
- **Estrutura Condicional Simples**
- Estrutura Condicional Composta
- Estrutura Condicional Encadeada
- Estrutura Condicional de Múltipla Escolha



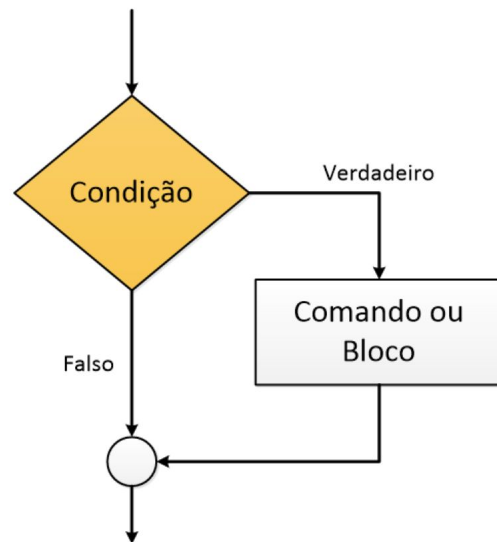
Estrutura Condicional

Existem diversas formas de representar o fluxo do programa com uma estrutura condicional.

Forma de Representação: Algoritmo

```
se condição então  
    comando1  
fimse
```

Forma de Representação: Fluxograma

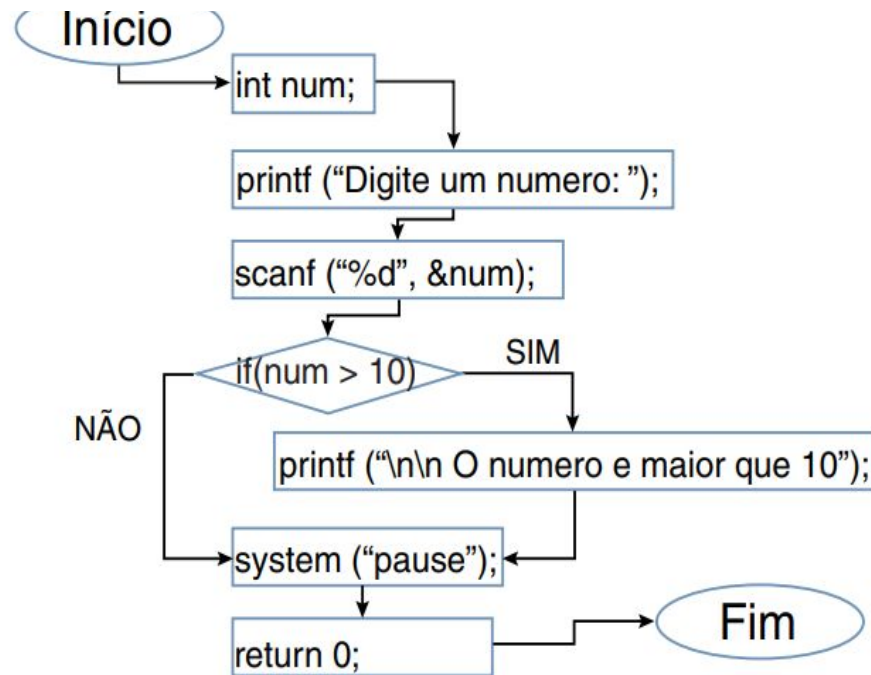




Estrutura Condicional

Decisões (Condicionais) :Permitem que o programa tome decisões baseadas em condições.

Na linguagem **C**, o comando **if** é utilizado sempre que é necessário escolher entre dois caminhos dentro do programa ou quando se deseja executar um ou mais comandos que estejam sujeitos ao resultado de um teste.





Estrutura Condicional

Decisões (Condicionais) :Permitem que o programa tome decisões baseadas em condições.

```
if (condição) {  
    # bloco se condição  
    for verdadeira;  
}  
  
if (condiçãoII) {  
    # bloco se a segunda  
    for verdadeira;  
}  
  
# restante do programa
```

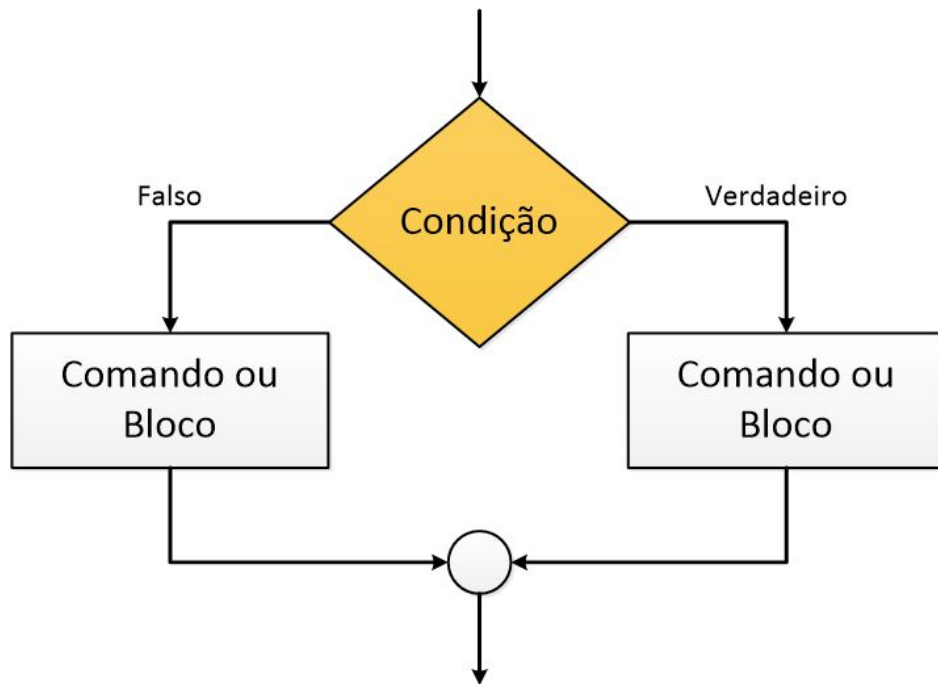
```
#include <stdio.h>  
int main() {  
    int idade = 18;  
    if (idade >= 18) {  
        printf("Você é maior de idade.\n");  
    } if (idade < 13) {  
        printf("Você é criança.\n");  
    }  
    return 0;  
}
```



Estrutura Condicional Composta

Forma de Representação:

Fluxograma



Algoritmo

se condição
então
 comando1
senão
 comando2
fimse

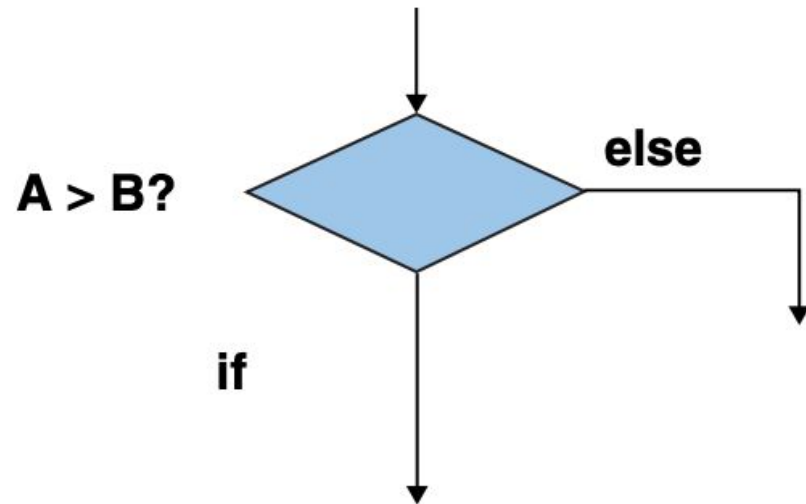
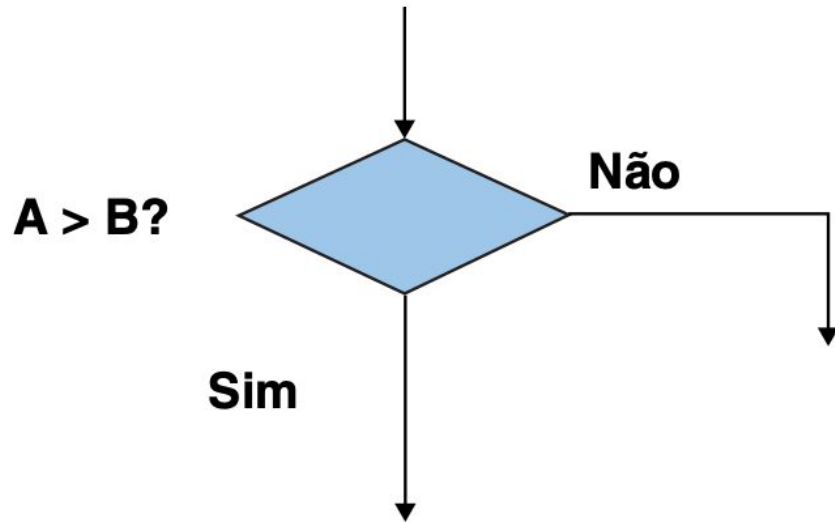


Estrutura Condicional Composta

Decisões (Condicionais) : Permitem que o programa tome decisões baseadas em condições.

```
if (condição) {  
    # bloco se condição  
    for verdadeira;  
} else if (condição) {  
    # bloco se a segunda  
    for verdadeira;  
} else {  
    # bloco se nenhuma  
    condição for verdadeira;  
}
```

```
#include <stdio.h>  
int main() {  
    int idade = 18;    if (idade >= 18) {  
        printf("Você é maior de idade.\n");  
    } else if (idade > 12) {  
        printf("Você é adolescente.\n");  
    } else {  
        printf("Você é criança.\n");  
    }  
    return 0;  
}
```





Testando 123..

Ler um n° inteiro e, se ele for positivo, informar seu inverso. Caso contrário, informar o seu quadrado.

Ler um n° e informar se ele é par ou ímpar.

Ler um n° e, se ele for um quadrado perfeito, informe sua raiz.

Faça um programa que leia do teclado uma letra, e imprima a sua correspondência no "Abecedário da Xuxa"

			
auto	else	long	switch
break	enum	register	typedef
case	extern	return	union
char	float	short	unsigned
const	for	signed	void
continue	goto	sizeof	volatile
default	if	static	while
do	int	struct	_Packed
double			



OBRIGADO

daciofmf@gmail.com